

BitCraft data sources and subscription reference

Prepared for Boss and the Space BitCraft tools. Updated October 7 2026.

We will build around a shared collector that maintains the public game data our tools need. Relay subscriptions supply current state; our database assembles useful views; provider APIs supply history, enriched lookups, and recovery checks. We will choose one primary source for each dataset rather than merging two overlapping feeds without an authority rule.

This reference separates the installed subscription foundation from optional experiments. Use the B and F identifiers to request a feature: for example, "Add F05 for my selected player" or "Start F09 with price alerts in Solmere."

Implementation status: the local Space collector now subscribes natively to BITCONNECT trading state in all nine current regions and BitCraftSync XP for watched players. Inventory and craft changes trigger enriched API refreshes; house/claim joins and history still use APIs. Optional F cards remain separate choices, except the basic activity/stamina context now shown in XP Tracker. This is a local implementation, not a deployment or world-scale capacity guarantee.

What we have today

Area	Working now	Boundary
Market	Durable item and cargo books; world and regional summaries; global sorting before pagination; item searches queue refreshes	BITCONNECT native order state; BitJita APIs retain directory, item checks and history. Coverage and freshness remain visible
Barter	Searchable exchanges, comparison rows, quantities, and fill estimates	BITCONNECT native player barter stalls and trade rows; shared APIs are retained for startup recovery and enrichment
Player tools	Collected skills, inventories, housing, selected claim storage, and craft records where supported	Watched players and claims, not a complete world inventory mirror
Activity and XP	Native watched XP, session totals, goals, character tabs, popout and OBS modes	Bounded history; source changes and gaps over two minutes restart the rate baseline
Nearby resources	Counts and nearest coordinates from a decoded 400 by 400 resource window	Periodic snapshots near the selected player's last known position; no interactive live map yet
Reference data	Public item definitions, recipes, skills, guides, and mirrored assets	Definitions and assets have their own update cycle

Region policy: include Virexal 7, Solmere 8, Marowik 9, Elyndor 12, Hexalis 13, Lumethis 14, Draxen 17, Oryxen 18, and Zephra 19. Southern Islands 3, Western Islands 11, Eastern Islands 15, and Northern Islands 23 are excluded even when provider directories still list them.

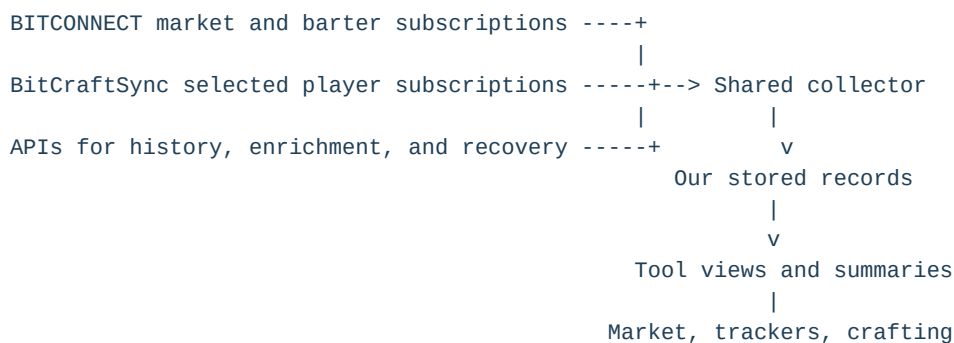
Reading the cards: Current means supported in today's tools. Foundation means part of the installed shared data layer; each card describes its exact remaining boundary. Later means an optional extension. Small, Medium, and Large describe relative scope and engineering work, not measured RAM requirements. Every example is an illustrative internal record, not an exact provider payload, real transaction, or live player report.

The shared foundation at a glance

Card	Dataset	Intended source and scope	What we gain
B01	World market orders	BITCONNECT; buy and sell state in all nine regions	Full order coverage, local comparisons, cancellation updates
B02	Barter exchanges	BITCONNECT trade, barter stall and scoped building state; all nine regions	Exact offered and required bundles with remaining stock
B03	Items and recipes	One selected catalog source; global data where available	Names, icons, units, recipes, and shared definitions
B04	Claims and players	Selected regional reference tables plus API enrichment	Readable owners and locations instead of anonymous entity IDs
B05	Watched player state	BitCraftSync; selected players and related containers	Native XP plus inventory change invalidation for enriched APIs
B06	Watched crafts	BitCraftSync; selected player-owned jobs	Craft change invalidation for enriched APIs; broader claim subscriptions remain optional

The division between providers is an initial recommendation. Both relays exposed the same market table names and returned matching order-row counts in our tests. Either can be a market source; actual failover must verify completeness and establish a new source generation before publishing results. Matching counts alone do not prove identical contents.

How the data reaches a tool



The collector owns external connections. Browsers subscribe to our database and receive the records needed for their current page. One person paging through Market does not create another full-world relay download.

Optional feeds at a glance

All F cards are optional. A working snapshot in today's tools does not mean the expanded feature on the card already exists.

Card	Optional feature	Scope	Example reason to add it
F01	Storage ledger and stock history	Medium	See who added materials and how stock changed
F02	Colony production planning	Medium	Coordinate inputs, jobs, outputs, and missing supplies
F03	Empire and claim relationships	Medium	Compare settlements belonging to one empire
F04	Live session companion	Small per player	Display stamina, buffs, action timers, and freshness
F05	Live nearby resource map	Medium per player	Show resources disappearing or appearing around me
F06	Terrain and elevation views	Large at world scale	Plan building sites or inspect paving and water
F07	Resource routes and area search	Medium to Large	Plan a gathering circuit through selected tiles
F08	Hexite deposit watch	Medium	Track deposits with known growth information
F09	Market alerts and opportunities	Small to Medium	Notify me when a regional offer meets my price
F10	Event history and session analysis	Medium to Large	Build our own history from trades and crafts we observe
F11	Public chat and announcement search	Medium	Find recent public trade messages by item
F12	Full world inventories and locations	Very Large	Support a specific feature that needs world-scale joins

Useful starting choices: F09 reuses the installed market foundation with relatively little new ingestion. F04 and F05 improve one selected player's experience. F01 and F02 are useful if settlement planning is the priority. F06 and F12 require a separate capacity decision.

Foundation world trading

B01 World market orders

Status: Installed native subscriptions with API directory, targeted checks and history. **Scope:** Medium, all nine current regions.

Provides: Orders identifying the item or cargo, buying or selling side, quantity, price threshold, owner, and claim. Installed ingestion uses `sell_order_state` and `buy_order_state`, with item and claim references to turn raw rows into readable offers. Both relays supplied these state tables in the measured probes. [S01, S02, S09]

```
{ "region": "Solmere", "item": "Rough Plank",  
  "side": "sell", "unitPriceHex": 3,  
  "quantity": 120, "claim": "Mossbank" }
```

Tool example: "Show every seller in Solmere, sort by unit price, and estimate buying 100 planks." The result uses every matching stored order before pagination. Claims with no suitable offer do not become stock.

We assemble: Claim names, item metadata, best prices, quantities, regional summaries, and fill estimates. Inserts, updates, and deletions affect the same canonical order store. API item refreshes remain a targeted verification path; history comes from separate endpoints.

Choose it when: Market freshness and comprehensive regional comparisons matter. This is the shared subscription foundation. **Boundary:** A successful snapshot proves coverage at that moment, not guaranteed zero lag or a globally simultaneous snapshot.

B02 Barter exchanges

Status: Installed native player barter ingestion; API recovery retained. **Scope:** Medium, all nine regions.

Provides: `trade_order_state` contains offered and required item/cargo components and remaining stock. `barter_stall_state` identifies player shops; `scoped_building_state`, `building_nickname_state` and claim joins provide labels. `marketplace_state` is a separate coin-market reference. Traveler recipe rows are excluded; valid one-sided donations/collection offers are retained. A building constructor is not assumed to be its current owner. [S09]

```
{ "claim": "Mossbank", "remainingTrades": 6,  
  "give": [{ "item": "Clay", "quantity": 10 }],  
  "receive": [{ "item": "Rough Plank", "quantity": 5 }] }
```

Tool example: "I need 20 planks; which barter offers can fill that, and what will they cost in clay?" Four trades require 40 clay in this illustrative exchange.

We assemble: Exact bundles, claim and owner labels, exchange availability, comparable quantities, and partial fills. We preserve mixed costs instead of pretending every trade has a single coin price.

Choose it when: Barter needs the same complete regional browsing as Market. **Boundary:** The 40,214 Solmere trade rows in the probe are raw table rows, not a verified count of available player barter offers. Stocks and applicable exchanges must be resolved before display.

Foundation references

B03 Item and recipe references

Status: Current catalogs and assets; Foundation refresh coordination. **Scope:** Small to Medium, shared definitions.

Provides: Definition tables and catalog endpoints supply item/cargo names, categories, icons, and recipe metadata. The regional schema includes candidates such as `crafting_recipe_desc`, `skill_desc`, and extraction definitions. Select a catalog source and verify its global/regional placement when generating bindings. [S04, S09]

```
{ "item": "Rough Plank", "kind": "item",  
  "category": "Wood", "recipeInputs": [  
    { "item": "Example Log", "quantity": 2 }  
  ] }
```

Tool example: A market row, recipe tree, inventory stack, and guide card all display the same item name and icon. The quantities above are illustrative, not a game recipe.

We assemble: Stable item keys, consistent item/cargo separation, recipe joins, and definition revisions. Existing mirrored assets stay on their own refresh path; downloading textures is not a live order subscription.

Choose it when: Any subscription needs readable items or calculations. It belongs in the foundation.

Boundary: Static definitions change with game updates. Large entity IDs remain strings or 64-bit values rather than JavaScript numbers.

B04 Claim and player references

Status: Current API lookup and enrichment; Foundation shared references. **Scope:** Medium, selected regional tables.

Provides: `claim_state`, `claim_local_state`, and `player_username_state` support claim, owner, and username joins. BitCraftSync's enriched claim API is a useful alternative to implementing every join immediately. [S03, S09]

```
{ "claim": "Mossbank", "region": "Solmere",  
  "owner": "Example Player", "supplies": 4200,  
  "position": { "x": 100, "z": 200 } }
```

Tool example: Click a market claim and see its identity, location, and relevant offers. Basic settlement information can enrich crafting results too.

We assemble: References used by market and barter, region-scoped claim IDs, and display labels. Names alone are not identity keys. Additional tier, upkeep, and membership views are separate enrichment choices.

Choose it when: Offers must link to real claims and owners. **Boundary:** A table appearing in the schema does not establish that every provider replicates it for every region; verify subscription acceptance and applied snapshots. Unavailable enrichment stays unknown instead of removing a valid order.

Foundation watched tools

B05 Watched player state

Status: Native watched XP and raw inventory change signals; enriched inventory, housing and claim APIs retained. **Scope:** Medium per watched cohort.

Provides: Candidate raw tables include `experience_state`, `inventory_state`, `housing state`, and `username/player` references. Container ownership and housing joins are nontrivial; BitCraftSync's player inventory, housing, and skill APIs can continue supplying enriched snapshots while raw subscriptions are added selectively. [S03, S08, S09]

```
{ "player": "Example Player", "skill": "Carpentry",  
  "xp": 125000, "held": { "Rough Plank": 80 },  
  "coverage": ["personal bags", "selected house"] }
```

Tool example: Inventory and Crafting share held quantities; Activity and Hunting share skill XP. A change updates all relevant tools without each tool polling independently.

We assemble: Safe inventory scopes, totals, selected-player subscriptions, and source-aware XP samples. A source switch, reconnect gap, or backward XP value must not create fake XP gain.

Choose it when: Multiple tools watch the same players. **Boundary:** Unknown inventory is not zero inventory; partial containers are not complete possessions. Start with selected players, not every player's possessions worldwide.

B06 Watched craft state

Status: Native selected player-owned craft change signals; enriched craft snapshots retained. Claim-wide native job subscriptions are later scope. **Scope:** Medium.

Provides: `progressive_action_state`, `passive_craft_state`, recipe definitions, buildings, and ownership references are the candidates for craft views. Enriched craft APIs remain useful during migration. [S03, S09]

```
{ "claim": "Mossbank", "output": "Example Boards",  
  "progress": 40, "totalProgress": 100,  
  "state": "in progress", "owner": "Example Player" }
```

Tool example: Passive Crafts and Open Crafts display the same progress record; Crafting shows what an observed job will produce. Completion changes the current job view.

We assemble: Job identity, recipe outputs, progress, owner and claim labels, and source timestamps. An ETA is only available if a trusted duration or progress rate exists; raw progress alone does not establish it.

Choose it when: Existing craft tools need fresher shared state. **Boundary:** Active jobs are not a permanent history of completed crafts. An all-world production planner is a later expansion, described in F02.

Later storage and production

F01 Storage ledger and stock history

Status: Later; storage snapshots already exist for selected scopes. **Scope:** Medium, selected chests or claims.

Provides: Storage-log records can associate item movement with a player and storage location. BitCraftSync exposes /storage-logs; an equivalent raw implementation would join storage_log_state with names and buildings. [S03]

```
{ "claim": "Mossbank", "storage": "Workshop Chest",  
  "player": "Example Player", "action": "deposit",  
  "item": "Rough Plank", "quantity": 40 }
```

Tool example: "Where did our 200 planks go?" Search observed deposits and withdrawals, then compare with stored stock snapshots.

We assemble: A retained ledger, filtering, stock change graphs, and explicit gaps. Movement records do not themselves prove consumption by a particular recipe.

Add when: You want settlement accountability or material-flow analysis. **Leave for later when:** Current quantities answer the question. The provider describes roughly 15-16 days of upstream storage-log retention; longer history requires our own retention and cannot recover records already absent upstream.

F02 Colony production planning

Status: Later; expands B05 and B06. **Scope:** Medium, a chosen set of claims.

Provides: Combine stock, progressive/passive jobs, and recipe definitions. This is primarily our derived model, not a single ready-made relay feed.

```
{ "target": "Example Boards", "required": 100,  
  "held": 20, "observedJobsOutput": 30,  
  "uncovered": 50 }
```

Tool example: "Make 100 boards for the settlement; what do we own, what is underway, and what should we buy?" Expand the uncovered amount through the chosen recipes and consult Market or Barter.

We assemble: Recipe choices, job outputs, shared-stock scope, material reservations, and planning totals. To avoid double counting, the planner must distinguish gross ingredients from inventory-aware allocations and explain whether supplies are merely observed or reserved.

Add when: Several players or claims coordinate production. **Leave for later when:** A single recipe calculator is sufficient. Observation cannot reserve game materials; any reservation we add is our planning record and does not execute a game action.

Later claims and session state

F03 Empire and claim relationships

Status: Later expansion; basic empire filtering already exists. **Scope:** Medium.

Provides: Candidates include `empire_state`, `empire_settlement_state`, empire nodes, ranks, and claim-member tables. Inspect actual relationships before deriving membership or territory. API empire/claim lookup can resolve a requested scope while we build those joins. [S09]

```
{ "empire": "Example Empire", "claims": [  
  { "name": "Mossbank", "region": "Solmere" },  
  { "name": "Stonewatch", "region": "Elyndor" }  
] }
```

Tool example: Compare offers and production across an empire's selected settlements, or identify which members have relevant skills.

We assemble: Time-stamped membership, role interpretation, claim groups, and optional territory overlays. Membership, ownership, citizenship, and empire affiliation are separate concepts.

Add when: Region filtering is too broad for your community. **Leave for later when:** A short list of claims suffices. The existence of empire tables is verified in the inspected regional schema; a complete authoritative cross-region relationship model is not implemented yet.

F04 Live session companion

Status: Basic activity and stamina context installed in XP Tracker through shared session snapshots; full buff/target companion HUD remains optional. **Scope:** Small per player.

Provides: BitCraftSync's `/bitme/session/<player-id>` bundles last-known position, stamina, buffs, actions, and selected target observations. Its intended cadence is about one request per second while the companion is open. [S05]

```
{ "player": "Example Player", "signedIn": true,  
  "stamina": { "current": 72, "max": 100 },  
  "action": "gathering", "positionAgeMs": 900 }
```

Tool example: A companion panel shows a gathering timer, stamina, and active buffs alongside Tool Rates or Hunting.

We assemble: Countdown display, last-known versus current state, freshness labels, and reconnect handling. A future raw-table subscription is useful only if it replaces these joins economically.

Add when: You want moment-to-moment context for your selected player. **Leave for later when:** XP and inventory snapshots are enough. A stored position does not prove the player is online; stamina regeneration and expiry countdowns need their own calculation rules.

Later live resources and terrain

F05 Live nearby resource map

Status: Later map and deltas; nearby counts already exist. **Scope:** Medium per selected player.

Provides: A BitCraftSync BMR1 resource window, a region dictionary, and optional BMD1 change frames from `/bitme/session/<player-id>/resources/ws`. The window is 400 by 400 tiles and 320,024 bytes. [S05]

```
{ "region": "Solmere", "windowTiles": [400, 400],  
  "nearby": [{ "resource": "Example Ore",  
    "tile": [100, 200], "distanceTiles": 12 }] }
```

Tool example: Display a nearby map; update a tile when a resource disappears or appears; highlight the nearest node of the selected type.

We assemble: A map renderer, decoded dictionary, resource anchors, distance, and incremental convergence. Count resource origins rather than every occupied footprint tile.

Add when: Nearby counts need visual context. **Leave for later when:** Nearest coordinates are enough. Refetch the window on resync, movement, reconnect, or dictionary changes. The documented 2,048-stream fleet cap applies specifically to this resource WebSocket, not to every relay subscription.

F06 Terrain and elevation views

Status: Later. **Scope:** Small windows first; Large for whole regions.

Provides: BitCraftSync offers world-window elevation through BME1 and regional roads maps containing terrain and overlays. The documented full regional protobuf is approximately 280 MiB; nine such exports are about 2.46 GiB before decoding or additional state. [S05, S06]

```
{ "tile": [100, 200], "elevation": 22,  
  "water": false, "paving": "Example Road",  
  "claim": "Mossbank" }
```

Tool example: Inspect an area for a building site, show claim boundaries, or compare elevations along a route.

We assemble: Coordinate conversion, rendering, region boundaries, terrain generations, and optional planning overlays. Start with requested windows or regions and use change validators where available.

Add when: You want a map or building planner. **Leave for later when:** Market and trackers are the priority. Those snapshot sizes are documented export sizes, not measurements of our finished application's RAM. Fetching one region's complete terrain does not describe all data in that region.

Later routes and deposits

F07 Resource routes and area search

Status: Later. **Scope:** Medium for requested areas; Large for world indexing.

Provides: The BitCraftSync roads resource lookup accepts requested hex tiles and returns resource nodes. Its documented request maximum is 16,384 tiles. Combine this with resource definitions and optional terrain data. [S07]

```
{ "resource": "Example Ore", "routeTiles": [
  [100, 200], [105, 203], [110, 208]
], "distanceTiles": 24 }
```

Tool example: "Find a gathering loop through this area with ore and trees, ending near our claim." First return candidate nodes; then estimate travel separately.

We assemble: Requested-area caching, node deduplication, route scoring, map conversion, and user constraints. Tile distance is not actual travel time; water, obstacles, interiors, and movement rules need explicit treatment.

Add when: Gathering routes are worth optimizing. **Leave for later when:** The nearest single node answers the need. A targeted area lookup is a practical first experiment before storing a complete world resource index.

F08 Hexite deposit watch

Status: Later. **Scope:** Medium.

Provides: BitCraftSync's deposit cache joins claims, resources, growth information, and locations. `/deposits` can be scoped by region. Active, depleted, and unresolved records need distinct handling. [S03]

```
{ "deposit": "Example Deposit", "region": "Solmere",
  "state": "depleted", "respawnAt": "<known timestamp>",
  "locationKnown": true }
```

Tool example: Keep a watch list of deposits and show which have a known growth deadline. Add a reminder based on that observed deadline.

We assemble: Region filtering, deduplicated deposits, growth interpretation, and reminders. Store when the observation was made and whether the growth/location join succeeded.

Add when: Deposit monitoring is useful to your group. **Leave for later when:** General nearby resources are sufficient. A timer is an observation or estimate, not a guaranteed future harvest; a missing growth join must stay unknown rather than becoming "ready now."

Later market alerts and events

F09 Market alerts and opportunities

Status: Later; mainly derived from B01 and B02. **Scope:** Small to Medium.

Provides: Our normalized order changes can drive price, quantity, claim, and barter conditions. API summaries or historical averages can add context when needed. New raw tables may not be necessary.

```
{ "item": "Rough Plank", "region": "Solmere",  
  "condition": "sell price at most 3 hex",  
  "minimumQuantity": 100, "matched": true }
```

Tool example: "Tell me when a claim has at least 100 planks at my price." Or compare a sell offer with buyers in another region after accounting for quantity and travel.

We assemble: Saved rules, deduplication, cooldowns, freshness checks, and delivery preferences. Evaluate each rule against complete applicable coverage, not a half-loaded region.

Add when: You repeatedly search for the same opportunities. **Leave for later when:** Manual browsing is enough. A price spread is not guaranteed profit: stock, cargo packaging, fees, travel, and stale offers affect the result. External notification delivery would be a separately configured feature.

F10 Event history and session analysis

Status: Later. **Scope:** Medium to Large, depending on retention.

Provides: BitCraftSync documents v2 event-table delivery, including market trades, craft events, and player deaths. Event records are received as they occur, without an initial historical snapshot or replay after disconnection. BITCONNECT event/protocol compatibility needs a separate test; our current probes used v1 state subscriptions. [S02]

```
{ "event": "trade observed", "item": "Rough Plank",  
  "quantity": 20, "unitPriceHex": 3,  
  "coverage": "connected observation interval" }
```

Tool example: A session report compares observed XP gain, craft events, and market activity over a selected interval.

We assemble: Event persistence, source generations, retention, duplicate rules, and gap records. Historical API data may supplement some trade views, but it is not a universal replay of every event.

Add when: We need our own future history. **Leave for later when:** Current state and provider trade history suffice. Never interpret a missing event as proof that nothing happened while we were disconnected.

Later social data and full mirrors

F11 Public chat and announcement search

Status: Later. **Scope:** Medium, deliberately bounded retention.

Provides: The inspected regional schema includes public chat_message_state; the BitJita API also documents chat retrieval. Acceptance, channels, retention, and payload shape need verification before enabling a collector. [S04, S09]

```
{ "channel": "public trade", "message":  
  "Example Player offers rough planks near Mossbank",  
  "region": "Solmere", "observedAt": "<timestamp>" }
```

Tool example: Search recent public messages mentioning an item, then cross-check current Market or Barter offers.

We assemble: Search, channel labels, retention, duplicate suppression, and optional item matching. A message is a player's statement, not a validated order, price, or reservation.

Add when: Public announcements help find trades the order tables do not capture. **Leave for later when:** Order data is enough. Keep this as a clear opt-in dataset with visible retention, rather than treating social messages as another permanent catalog.

F12 Full world inventories and locations

Status: Deferred until a specific feature justifies it. **Scope:** Very Large.

Provides: Public inventory, location, resource, building, and ownership rows could support broad world joins. The source documentation describes location state as millions of rows per region and demonstrates selective location subscriptions instead of a full dump. [S03]

```
{ "query": "Compare selected settlement warehouses",  
  "scope": ["Mossbank", "Stonewatch"],  
  "coverage": "selected claims only" }
```

Tool example: A world logistics feature could analyze many settlements. The same first prototype can often work with selected claim APIs and much smaller retained scopes.

We assemble: A dedicated mirror, capacity planning, safe ownership joins, migration handling, and recovery checkpoints. Complete duplication from both providers also requires conflict and authority rules.

Add when: A concrete product needs these joins and measurements support the storage cost. **Leave for later when:** We simply want future flexibility. It is a separate infrastructure project, not an automatic extension of the market benchmark.

Speed cadence and sensible API use

The October 7 research probes used native v1 JSON subscriptions from this machine. Three Solmere order trials were run per provider; the nine-region order sweep was run once per provider, loading regions sequentially. Connections were closed after each probe.

Measured snapshot	BITCONNECT	BitCraftSync
Solmere buy and sell orders, three trials	0.34-0.87 seconds	0.49-0.72 seconds
All nine current regions, buy and sell orders	4.19 seconds	4.24 seconds
Order rows across those nine snapshots	56,206	56,206
Nine-region JSON snapshot payloads	5.65 MB	16.11 MB
Solmere orders plus marketplace and trade state	0.69 seconds	0.83 seconds

What these timings mean: Connection setup, snapshot receipt, and JSON parsing. They exclude our joins, database ingestion, indexes, rendering, and full-world barter or inventory loading. Row counts are summed snapshot rows, not a separately verified count of unique world orders. The ten-second market observation intervals recorded no relevant updates, so these probes did not measure game-change propagation latency. [S10]

Continuous feeds: Subscribe once, apply changes, and read our own database whenever a tool needs it. Installed trading publication is batched at least three seconds apart per changed region; selected XP is sampled every ten seconds, transport heartbeats every fifteen seconds, and the XP UI refreshes every ten seconds. HTTP enrichment and local database work can delay these targets. Initial import includes joins and summary writes and is slower than the order-only probe timings above. Load large subscription sets in stages. The public documentation does not establish a general numeric WebSocket quota for either provider. [S01, S02]

Use APIs where the result is already useful: BitJita for item-book checks and completed-trade history; BitCraftSync for enriched player/claim views and targeted map windows. BitJita documents 250 HTTP requests per minute. That is its API limit, not a demonstrated WebSocket allowance; the site shares its configured request identity and budget. [S04]

Bulk summaries are not full books: BitJita's bulk-price endpoint can summarize batches, but summary statistics cannot identify every claim or reconstruct all regional orders. It is optional enrichment, not a replacement for B01. [S04]

Using the new XP Tracker

Open `/bitcraft/activity/setup` for the full tracker. **Popout** opens a compact companion window; **Copy OBS URL** saves an opaque public widget profile and exposes a clean browser-source URL. The OBS view has a transparent page, selected panel opacity, and no editing or audio controls. Use the settings drawer for themes, goals, size and visible sections. OBS needs network access to this app; `space.test` works only on a machine that resolves that local hostname.

Track up to five characters in the full/popout interface. The selected character receives activity and passive-craft details; background characters sample XP only. Session XP is separate from lifetime XP. Active time counts observed earning windows, not all time the window is open. Unknown XP, backward values, source changes and long gaps do not create fake gains. XP/hour and ETA need a continuous sample window; they stay in Sampling while it fills. All tracker presentations are silent; there are no sound controls or audio alerts.

Operating bounds and recovery

One managed collector shares nine trading connections. It opens at most thirty selected-player BitCraftSync connections; additional watched players retain shared API paths. Each raw scope has a 300,000-row cap and a 64 MB frame cap; schema contracts are separately pinned for the two providers. A provider schema change fails that scope visibly until reviewed with `npm run bitcraft:relay-schema`. This is a deliberate compatibility safeguard.

Only a complete Applied snapshot is published. Reconnects replace the complete regional generation; malformed frames and failed writes retain stored results. API item checks may enrich metadata but cannot replace the native-owned regional order rows. Find item still queues its shared API check; history remains on demand. We do not automatically blend two providers during an outage. Stored native coverage remains authoritative until a reviewed replacement/release is implemented; disabling `BITCRAFT_NATIVE_RELAYS` does not erase that coverage or silently hand it back to rolling HTTP books.

`relay_status` exposes coverage, heartbeat, generation and errors; `relay_region` privately retains normalized regional snapshots. `npm run test:relay-server` checks authority and reconciliation in an isolated database. `npm run test:relay-live` probes real provider snapshots; `npx tsx scripts/relay-runtime-check.ts` checks the managed local app. Optional F01-F12 datasets are not enabled simply because a source table exists.

Data contracts and rollout

What our own data layer should retain

Each retained scope should identify the provider, database, region, collection generation, and snapshot/update time. It should distinguish received, validated, complete, delayed, and unsupported data. Our tool views must preserve unknown values and avoid mixing item and cargo IDs.

For a region, publish an initial snapshot only after all required subscriptions and joins are ready. Apply order removals as well as inserts. On reconnect, reconcile a complete regional snapshot before declaring coverage restored. Retain the last usable results during an outage, with an honest timestamp.

Choose one authority per dataset. If we switch providers, publish a validated replacement generation; do not let an older API response overwrite a later relay state simply because it arrived last. Provider timestamp meanings and cross-provider ordering must be established during integration.

Separate current state from history. Current rows can be replaced; observed events need deliberate retention and explicit gaps. The current generic store's 500 feed scopes, 100,000 projected entities, and six-hour bounded XP samples are not a capacity plan for full-world subscriptions. Dedicated market storage already sits outside the generic feed cap; wider replication needs its own sizing. [S11]

Implementation order

1. **Trading foundation:** B01-B04. Start one region, compare API and relay results, then cover all nine current regions. Verify cancellations, item/cargo distinctions, claim joins, reconnects, and global sorting.
2. **Existing player tools:** B05-B06. Subscribe selected players and claims. Verify container coverage, house joins, craft transitions, and XP continuity. Retain useful enriched API paths.
3. **First optional feature:** Choose F09, F04/F05, or F01/F02 according to what you want to use next. Set a bounded scope and a concrete acceptance example.
4. **Heavy datasets:** Consider F06 and F12 only after measuring snapshot bytes, decoded memory, steady update rate, and reconnect time for the chosen scope.

A quick request format

Feature: F05 Live nearby resource map

Scope: My selected player in Solmere

Question: Where is the nearest ore node?

Example: A map highlights the node and updates when it disappears

Keep: Current window and a short diagnostic log

Delivery: Inside Tool Rates; no external notifications

This lets us agree on a useful tool without first committing to collecting the entire world.

Sources and verification

Provider behavior was checked against these primary sources on October 7 2026. Source pages and upstream schemas can change. Candidate tables still need acceptance and completeness tests at implementation time.

- **S01 BITCONNECT documentation:** relay.bitjita.com. Public replica, connection surface, read-only behavior, and excluded-table refusals.
- **S02 BitCraftSync subscription tutorial:** [Subscribe to tables](#). Initial snapshots, staged loading, state updates, and v2 event limitations.
- **S03 BitCraftSync joined cache tutorial:** [Relay cache joins](#). Player/claim/craft/storage joins, deposit data, selective location handling, and storage-log retention.
- **S04 BitJita API documentation:** [Developer API](#). Item books, trade history, bulk summaries, chat, crafts, and HTTP quota.
- **S05 BitCraftSync companion tutorial:** [Bit Me API and maps](#). Session cadence, BMR1/BME1 windows, resource deltas, sizes, and convergence.
- **S06 BitCraftSync terrain format:** [Region map format](#). Terrain and overlay structure; full-region export sizing.
- **S07 BitCraftSync resource lookup:** [Hex resource lookup](#). Targeted tile requests and request bounds.
- **S08 BitCraftSync player examples:** [Extract player data](#). Username, player state, experience, and skill joins.
- **S09 Inspected schemas:** [BITCONNECT region 8](#) and [BitCraftSync region 8](#). Candidate table names and public access declarations; declaration alone does not prove live replication coverage.
- **S10 Local benchmark evidence:** `output/research/relay-flow-benchmark.json`, `relay-flow-benchmark.mjs`, and `bitconnect-market-probe.mjs`. Counts, timings, payload bytes, scope, and observation intervals.
- **S11 Current implementation:** `scripts/bitcraft-collector.ts`, `bitcraft/src/collection.ts`, `bitcraft/src/collection-schema.ts`, `bitcraft/src/market-store.ts`, `bitcraft/src/index.ts`, `src/bitcraft-ui/api.js`, and `README.md`. Native relay collection, shared API enrichment, storage bounds, watched scopes, durable market reads, and UI flows. Also see `scripts/bitcraft-relay.ts`, `scripts/bitcraft-relay-collector.ts`, `bitcraft/src/relay-trading.ts` and `src/bitcraft-ui/xpSession.js`.

Maintenance: Keep this source file alongside the code. When a card ships, change its status, record the supported scope and validation, and rebuild the PDF. A research probe should not change a feature's status to Current.